

Лекция 7: Файлы (Ввод/Вывод)

Введение

До сих пор наши программы работали только с данными, которые существовали во время выполнения: переменные, ввод с клавиатуры, вывод на экран. Но после завершения программы все эти данные исчезают. Чтобы сохранить информацию между запусками программы (результаты расчётов, настройки, пользовательские данные), необходимо использовать **файлы**.

Java предоставляет богатый набор классов для работы с файлами. Сегодня мы изучим основы файлового ввода-вывода:

- Пакеты `java.io` и `java.nio`
- Чтение из текстового файла
- Запись в текстовый файл
- Обработка исключений (файл не найден, ошибка доступа)
- Современный подход `try-with-resources`

1) Основы работы с файлами. Исключения

Пакеты для работы с файлами

Основные классы для текстовых файлов находятся в пакете `java.io`:

Класс	Назначение
<code>FileReader</code>	Чтение символов из файла
<code>BufferedReader</code>	Буферизованное чтение (эффективно, есть метод <code>readLine()</code>)
<code>FileWriter</code>	Запись символов в файл
<code>BufferedWriter</code>	Буферизованная запись
<code>File</code>	Представление пути к файлу или директории

Таблица 1. Работа с файлами в Java

Исключения (Exceptions)

При работе с файлами постоянно могут возникать ошибки:

- Файл не существует
- Нет прав на чтение/запись
- Диск заполнен
- Файл открыт другой программой

Поэтому методы работы с файлами требуют обработки исключений.
Основные исключения:

- `FileNotFoundException` — файл не найден
- `IOException` — общая ошибка ввода-вывода

`try-catch-finally` и `try-with-resources`

До Java 7 типичный код выглядел так:

```
FileReader fr = null;
BufferedReader br = null;
try {
    fr = new FileReader("file.txt");
    br = new BufferedReader(fr);
    // чтение
} catch (IOException e) {
    e.printStackTrace();
} finally {
    try {
        if (br != null) br.close();
        if (fr != null) fr.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

Это громоздко. Начиная с Java 7, появилась конструкция `try-with-resources`, которая автоматически закрывает ресурсы:

```
try (BufferedReader br = new BufferedReader(new FileReader("file.txt"))) {
    // чтение
} catch (IOException e) {
    e.printStackTrace();
}
```

Мы будем использовать **try-with-resources** — это современный и безопасный способ.

2) Запись в текстовый файл

FileWriter и BufferedWriter

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

public class WriteToFile {
    public static void main(String[] args) {
        String fileName = "output.txt";

        // try-with-resources автоматически закроет BufferedWriter
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(fileName)))
        {
            writer.write("Первая строка");
            writer.newLine(); // переход на новую строку (платформонезависимо)
            writer.write("Вторая строка");
            writer.newLine();
            writer.write("Число: " + 42);

            System.out.println("Данные успешно записаны в файл " + fileName);
        } catch (IOException e) {
            System.out.println("Ошибка при записи в файл: " + e.getMessage());
            e.printStackTrace();
        }
    }
}
```

Режимы записи: перезапись и добавление

Конструктор `FileWriter` может принимать второй параметр — флаг `append`:

- `new FileWriter("file.txt")` — перезаписывает файл (или создаёт новый)
- `new FileWriter("file.txt", true)` — добавляет данные в конец файла

java

```
public class FileAppend {
    public static void main(String[] args) {
        String fileName = "log.txt";
```

```

// Добавление в конец файла
try (BufferedWriter writer = new BufferedWriter(new FileWriter(fileName, true))) {
    writer.write("Новая запись: " + java.time.LocalDateTime.now());
    writer.newLine();
} catch (IOException e) {
    e.printStackTrace();
}
}
}

```

Запись с помощью PrintWriter (удобное форматирование)

PrintWriter имеет методы `print()`, `println()`, `printf()` как у `System.out`:

```

import java.io.PrintWriter;
import java.io.FileWriter;
import java.io.IOException;

public class PrintWriterDemo {
    public static void main(String[] args) {
        try (PrintWriter writer = new PrintWriter(new FileWriter("formatted.txt"))) {
            writer.println("Имя: Иван");
            writer.println("Возраст: 25");
            writer.printf("Сумма: %.2f руб.", 123.456);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

3) Чтение из текстового файла

BufferedReader и FileReader

Основной способ чтения текстового файла построчно:

```

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

```

```

public class ReadFromFile {
    public static void main(String[] args) {
        String fileName = "output.txt";

        try (BufferedReader reader = new BufferedReader(new FileReader(fileName)
        )) {
            String line;
            int lineNumber = 1;

            // readLine() возвращает null, когда достигнут конец файла
            while ((line = reader.readLine()) != null) {
                System.out.println("Строка " + lineNumber + ": " + line);
                lineNumber++;
            }
        } catch (IOException e) {
            System.out.println("Ошибка при чтении файла: " + e.getMessage());
        }
    }
}

```

Чтение всего файла в строку (Java 11+)

```

import java.nio.file.Files;
import java.nio.file.Path;
import java.io.IOException;

public class ReadAll {
    public static void main(String[] args) {
        try {
            String content = Files.readString(Path.of("output.txt"));
            System.out.println("Содержимое файла:\n" + content);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

Чтение всех строк в список (Java 8+)

```

import java.nio.file.Files;
import java.nio.file.Path;
import java.io.IOException;
import java.util.List;

```

```

public class ReadAllLines {
    public static void main(String[] args) {
        try {
            List<String> lines = Files.readAllLines(Path.of("output.txt"));
            for (int i = 0; i < lines.size(); i++) {
                System.out.println((i+1) + ": " + lines.get(i));
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

Проверка существования файла

```

import java.io.File;

public class FileExists {
    public static void main(String[] args) {
        File file = new File("data.txt");

        if (file.exists()) {
            System.out.println("Файл существует");
            System.out.println("Размер: " + file.length() + " байт");
            System.out.println("Читаемый: " + file.canRead());
            System.out.println("Записываемый: " + file.canWrite());
        } else {
            System.out.println("Файл не найден");
        }
    }
}

```

4) Копирование файлов и работа с путями

Копирование содержимого из одного файла в другой

```

java
import java.io.*;

public class CopyFile {
    public static void main(String[] args) {

```

```

String sourceFile = "source.txt";
String destFile = "destination.txt";

try (BufferedReader reader = new BufferedReader(new FileReader(sourceFile
));
    BufferedWriter writer = new BufferedWriter(new FileWriter(destFile))) {

    String line;
    while ((line = reader.readLine()) != null) {
        writer.write(line);
        writer.newLine();
    }
    System.out.println("Файл скопирован успешно");
} catch (FileNotFoundException e) {
    System.out.println("Исходный файл не найден: " + sourceFile);
} catch (IOException e) {
    System.out.println("Ошибка ввода-вывода: " + e.getMessage());
}
}
}

```

Простой способ копирования через Files (Java 7+)

```

java
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardCopyOption;
import java.io.IOException;

public class CopyFileNIO {
    public static void main(String[] args) {
        try {
            Files.copy(Path.of("source.txt"), Path.of("destination.txt"),
                StandardCopyOption.REPLACE_EXISTING);
            System.out.println("Файл скопирован (NIO)");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

Итоги седьмой лекции

Сегодня мы изучили:

Основы файлового ввода-вывода:

- Пакет `java.io`: `FileReader`, `FileWriter`, `BufferedReader`, `BufferedWriter`
- `try-with-resources` для автоматического закрытия ресурсов
- Исключения: `IOException`, `FileNotFoundException`

Запись в файл:

- `BufferedWriter` с методом `write()` и `newLine()`
- `PrintWriter` для удобного форматирования
- Режим добавления (`append = true`)

Чтение из файла:

- `BufferedReader` с методом `readLine()` (возвращает `null` при конце файла)
- `Files.readString()` и `Files.readAllLines()` для упрощённого чтения

Ключевые выводы:

1. Всегда закрывайте ресурсы (или используйте `try-with-resources`)
2. Обработывайте исключения — файл может не существовать
3. Для текстовых файлов используйте `BufferedReader/BufferedWriter` для производительности
4. Проверяйте существование файла перед чтением с помощью `File.exists()`